Automatic License Plate Recognition

Luca Granucci
Department of Information Engineering (DII)
University of Pisa
Pisa, Italy
l.granucci1@studenti.unipi.it

Lorenzo Marranini
Department of Information Engineering (DII)
University of Pisa
Pisa, Italy
l.marranini@studenti.unipi.it

***Abstract*—Automatic License Plate Recognition (ALPR) on the Chinese City Parking Dataset (CCPD) poses two challenges: localising plates under various conditions, and recognising the plates characters, which have both chines character and alphanumerical ones. We present a two-stage pipeline that addresses both. For plate detection, we train various YOLO11 models to predict the four plate corners directly. For recognition, we evaluate three models on the rectified crops: a position-classification CNN with seven independent heads, a CTC-based LPRNet port, and PaddleOCR (PP-OCRv5). All training stages use stratified 4-fold cross-validation over the entire working set, and we report end-to-end accuracy on a fresh CCPD subset not used in any prior training or validation. Our pipeline achieves 95.8% full-plate accuracy end-to-end, improving over the zero-shot model by 24.2%.**

I. Introduction

In recent years, Intelligent Transportation Systems (ITS) have become a cornerstone in the development of smart cities, traffic management, and automated law enforcement [1]. One of the challenges of such systems is Automatic License Plate Recognition (ALPR), which aims to detect and read vehicle license plates from images or video streams without human intervention.

Despite advancements in Machine Vision, ALPR remains a challenging task due to several real-world variability factors, including different lighting conditions, weather artifacts, camera angles, and varying vehicle speeds. To address these challenges, deep learning approaches, and specifically Convolutional Neural Networks are used [2]. Among these, the YOLO family of models [3] stands out for its single-stage detection.

In this paper, developed as the final project for the *Intelligent Systems* course at the University of Pisa, we present a complete end-to-end pipeline for ALPR on the Chinese City Parking Dataset (CCPD) [4]. We apply the Knowledge Discovery in Databases (KDD) methodology [5], starting from data selection and exploration, moving through model training with a k-fold cross-validation, and concluding with a performance evaluation on a fresh CCPD subset not used during training. The primary goal of this project is to implement an Intelligent System capable of reliable license plate detection and recognition, and to compare two design choices central to the pipeline: **(i)** for the detection stage, two different YOLO11 task formulations: keypoint regression (*pose*) versus instance segmentation (*seg*), both providing the four plate corners needed for perspective rectification; and **(ii)** for the recognition stage, three alternative recognizers: a position-classification CNN with seven independent heads, a CTC-based LPRNet [6], and a zero-shot PaddleOCR [7] baseline.

II. Brief state of the art analysis

The development of Automatic License Plate Recognition (ALPR) systems has spanned several decades. As outlined in the literature review by Du et al., early ALPR frameworks were traditionally broken down into four main stages: image acquisition, license plate extraction, character segmentation, and character recognition [8]. These classical solutions relied heavily on manual feature extraction and standard computer vision techniques. For example:

- Edge-based extraction methods could achieve around 96.2% accuracy by matching vertical lines.
- Color-based methods exploiting the HLS color model reached localization rates up to 97.9%.
- Texture-based approaches using Gabor filters achieved around 98% accuracy, though they were computationally expensive.

While these classical methods performed well in controlled environments, they suffered from a severe lack of robustness when subjected to real-world conditions like varying illumination, distorted plates, or complex backgrounds.

The paradigm shifted dramatically with the advent of Deep Learning [2]. Convolutional Neural Networks (CNNs) revolutionized the localization phase by learning complex hierarchical features directly from the data, moving away from hand-crafted rules. For the detection phase, efficient anchor-free models like YOLOX became a standard for edge applications. More recently, the state of the art has shifted toward Transformer-based architectures [3], [4]. These models use self-attention mechanisms to model global dependencies, successfully resolving issues like partial occlusion and perspective distortion.

As discussed by Lavezzini in his 2025 master's thesis, deploying these modern models on Edge devices requires a careful optimization of the accuracy-throughput trade-off [9]. By testing a modular three-stage pipeline, his work reported specific numerical results for these modern architectures:

- **Plate Detection:** The Transformer-based RT-DETR v2 (R50 variant) achieved a 90.0% mAP@50-95 and a 91.7% AR@100.

- **Character Recognition (OCR):** To overcome the data-inefficiency of pure Vision Transformers, a Compact Convolutional Transformer (CCT-S) was implemented. It achieved an impressive 99.1% plate accuracy and a Character Error Rate (CER) of just 0.17%, with a CPU inference time of only 14 ms.

III. Background

A. Use Case Scenarios

The proposed Intelligent System targets two complementary deployment scenarios. The first is *static*, mirroring the acquisition setting of the CCPD dataset itself: a fixed camera photographs a parked vehicle, and the system must return the plate string from a single still image. In this scenario latency constraints are mild. The second is *continuous video monitoring*, in which the system processes a live camera stream, must track each vehicle across frames. This scenario adds real-time throughput and temporal-consistency requirements.

B. System Architecture

The system is organized as a two-stage pipeline, illustrated in Fig. 1. Given an input image, Stage 1 localises the license plate and estimates the four corners of its quadrilateral; a perspective rectification step then warps the plate region into a rectangular crop. Stage 2 reads the seven-character plate string from the rectified crop using one of three alternative recognition engines. The two stages are fully decoupled: any detector that outputs four ordered corners can feed any of the recognisers, which allows each design choice to be evaluated independently

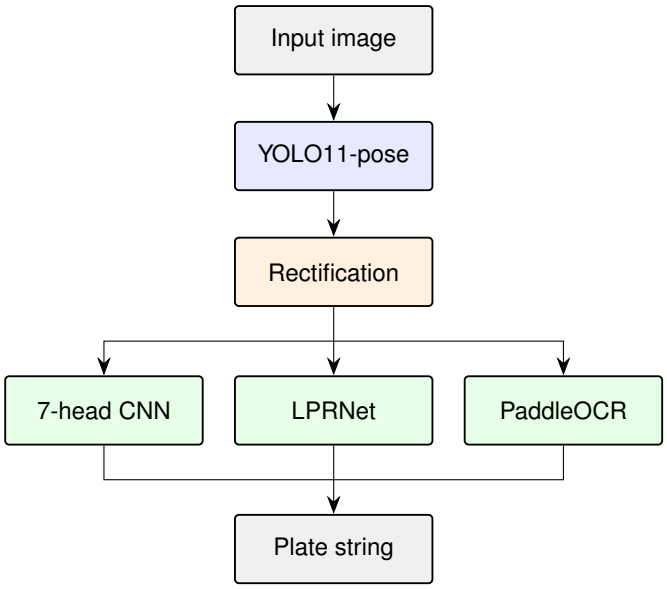

Fig. 1. Block diagram of the implemented pipeline

C. Stage 1: Plate Localization

For the localization stage we adopt YOLO11, the latest iteration of the single-stage YOLO detector family, comparing two instances, pose and segmentation, that both provide us with the plate orientation and tilt, not only its position.

1) Keypoint regression (pose): We configure the pose model with four keypoints per plate, corresponding to the corners of the plate in a fixed order (top-left, top-right, bottom-right, bottom-left).

2) Instance segmentation (seg): The segmentation variant predicts a binary mask of the plate region. The four corners are not an explicit output and must be recovered from the mask contour. Comparing the two formulations under identical training conditions is one of the experimental questions of this work.

For both formulations we evaluate the nano (n), small (s) and medium (m) capacity scales.

D. Perspective Rectification

CCPD images exhibit median tilt angles above 20° in the `Rotate` and `Tilt` subsets. Since these conditions degrade the performances of the recognizers, we rectify every detected plate before reading it. Let $\mathbf{p}_i = (x_i, y_i)$, $i = 1, \ldots, 4$, be the four predicted corners ordered clockwise from the top-left, and let $\mathbf{p}'_i$ be the corners of the target rectangle of size $W \times H$ (we use 168×48 pixels, matching the 3.5:1 aspect ratio of Chinese plates). The homography $\mathbf{H} \in \mathbb{R}^{3\times3}$ is the unique (up to scale) projective transform satisfying

$$\lambda_i \begin{bmatrix} \mathbf{p}'_i \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} \mathbf{p}_i \\ 1 \end{bmatrix}, \qquad i = 1, \ldots, 4, \tag{1}$$

and is obtained in closed form from the four point correspondences. Warping the input image by $\mathbf{H}$ yields a fronto-parallel, horizontally aligned crop in which character shape and spacing are normalized regardless of the original viewpoint. To quantify the benefit of this step, all recognizers are also trained and evaluated on plain bounding-box crops (*box* vs. *warp*, Section V).

E. Stage 2: Character Recognition Engines

The recognition stage receives the rectified crop, resized to 94×24 pixels, and outputs a seven-character string over the CCPD vocabulary of 67 classes: 33 ideographs (the 31 provincial abbreviations plus two special characters, one for police plates and one for learner plates), 24 Latin letters, and 10 digits. A blank token is added when training the CTC-based recogniser, giving the 68-class CTC vocabulary used by LPRNet.

1) Multi-head position classifier (7-head CNN): The first engine treats plate reading as seven independent classification problems, one per character slot. A shared convolutional backbone of three Conv–BatchNorm–ReLU–MaxPool blocks (32, 64 and 128 channels) is followed by global average pooling, producing a single 128-dimensional descriptor of the whole crop; seven parallel linear heads then each predict one character over the shared 67-class vocabulary. With $\sim$0.16 M parameters this is by far the lightest model evaluated.

2) LPRNet: The second engine is our PyTorch port of LPRNet, a segmentation-free sequence recogniser designed specifically for license plates. The backbone interleaves *small basic blocks*, bottleneck units factorising a 3×3 convolution into 3×1 and 1×3 branches, with max-pooling layers that downsample width more aggressively than height, so that the final feature map preserves a fine-grained horizontal axis aligned with the character sequence. Feature maps from four depths of the backbone are normalised, pooled to a common resolution and concatenated, providing the classifier with multi-scale context. A 1×1 convolution maps the concatenated features to per-timestep logits over the 68-class CTC vocabulary (the 67 plate characters plus a blank token), and the height dimension is averaged out, yielding a sequence of class distributions along the plate width. The network is trained end-to-end with the Connectionist Temporal Classification (CTC) loss, which marginalises over all monotonic alignments between the output sequence and the 7-character ground truth:

$$\mathcal{L}_{\mathrm{CTC}} = -\log \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{y})} \prod_{t=1}^{T} p(\pi_t \mid \mathbf{x}), \tag{2}$$

where $\mathcal{B}$ collapses repeated symbols and removes blanks. At inference, greedy decoding selects the most probable class at each timestep and applies $\mathcal{B}$. Because no explicit character segmentation is required, the model tolerates small residual misalignments left by imperfect rectification.

3) PaddleOCR (zero-shot baseline): The third engine is PP-OCRv5, the recognition model of the PaddleOCR toolkit [**?**], used *zero-shot*: a general-purpose OCR system trained on broad multilingual text, applied to plate crops without any fine-tuning on CCPD. It serves as baseline that answers a practical question: how much accuracy does a domain-specific, locally trained recogniser actually buy over an off-the-shelf OCR engine?

IV. Dataset

To train, validate and test the models discussed in the following sections, we used the Chinese City Parking Dataset (CCPD) [4], a public benchmark for Chinese license plate recognition. The dataset comprises 351,977 images of vehicles captured by parking-fee collectors in a single provincial capital. Each image has a fixed resolution of 720×1160 pixels and contains exactly one license plate. Annotations are encoded in the filename rather than in separate label files, and provide for every image: the plate's bounding box, the four corner coordinates of the plate quadrilateral, the seven-character ground-truth string, and two scalar image-quality descriptors (brightness and blur). An example image and its decoded labels are shown in Fig. 2. CCPD is organised in eight named subsets, each isolating a different failure mode encountered during acquisition. Their composition is summarised in TABLE I. The `Base` subset, which represents the standard frontal-plate acquisition condition, accounts for the majority of the dataset (199,996 images, $\sim 57\%$); the remaining seven subsets isolate specific adverse conditions including extreme

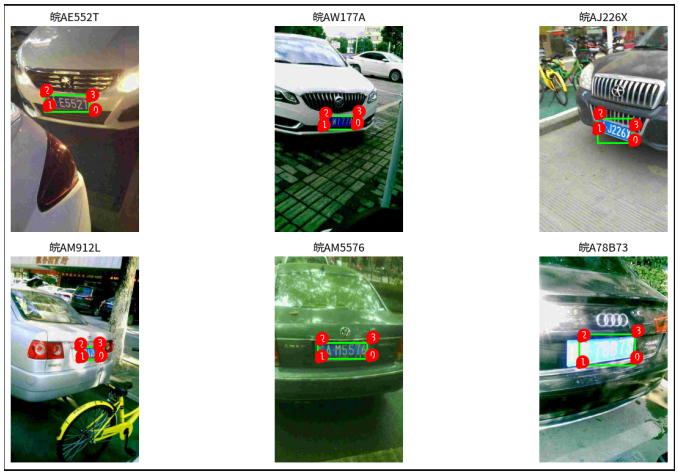

Fig. 2. Six representative CCPD images annotated from the encoded filename labels.

rotation (`Rotate`, 10,053 images, median horizontal tilt 22°), tilt (`Tilt`, 30,216 images, median vertical tilt 31°), low brightness (`DB`, 10,132 images, median brightness 28 against the 113 of `Base`), motion blur (`Blur`, 20,611 images, median blur score 4), challenging illumination (`Challenge`), distant or near plates (`FN`), and adverse weather (`Weather`). This organisation allows us to report not only an aggregate accuracy figure but also a per-condition breakdown, which is essential for characterising the robustness of the proposed pipeline.

TABLE I
CCPD SUBSET COMPOSITION AND MEDIAN VALUES OF THE IMAGE-QUALITY DESCRIPTORS ENCODED IN THE DATASET.

Subset	**N**	**Tilt$_h$**	**Tilt$_v$**	**Bright.**	**Blur**
`Base`	199,996	90	87	113	37
`Challenge`	50,003	3	3	105	12
`Tilt`	30,216	21	31	95	76
`FN`	20,967	5	9	101	33
`Blur`	20,611	2	5	76	4
`DB`	10,132	3	9	28	11
`Rotate`	10,053	22	12	115	68
`Weather`	9,999	2	4	80	41
Total	**351,977**				

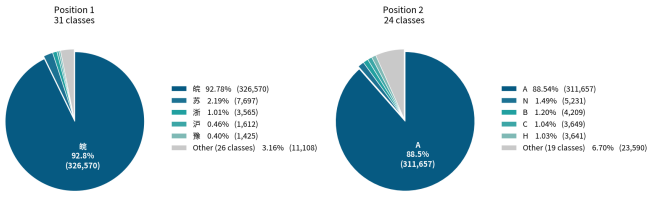

Fig. 3. Distribution of characters in the first two positions

The character vocabulary of CCPD reflects the structure of Chinese license plates: position 1 is a provincial ideograph (31 provinces are represented in the data, Fig. 3), position 2 is one of 24 Latin letters (excluding `I` and `O`, which are not

used in Chinese plates to avoid confusion with the digits `1` and `0`), and positions 3–7 are an alphanumeric mix of the same 24 letters and the 10 digits. Counting also the two special ideographs reserved for police and learner plates, the full vocabulary comprises $31 + 2 + 24 + 10 = 67$ character classes, to which a blank token is added when training CTC-based recognizers (Section III-E), giving the 68-class CTC vocabulary. Positions 1 and 2 are highly imbalanced: *Anhui*, the province where data collection took place, accounts for 326,570 of the 351,977 plates ($\sim$93%), and the letter `A` dominates position 2 in similar proportion, since Anhui's plates predominantly originate from a single municipality. The five least frequent province classes are seen in fewer than 30 images each. Positions 3–7, on the contrary, follow distributions much closer to uniform, as is typical of real-world plate sequences. Within these positions, however, digits (`0--9`) appear noticeably more frequently than letters (`A--Z` excluding `I`, `O`), and the digit-to-letter ratio grows from position 3 to position 7, reflecting the convention by which Chinese plate serial codes are predominantly numeric and use letters more sparingly. The class distributions of positions 1 and 2 are shown in Fig. 3; the remaining positions are illustrated in Fig. 4.

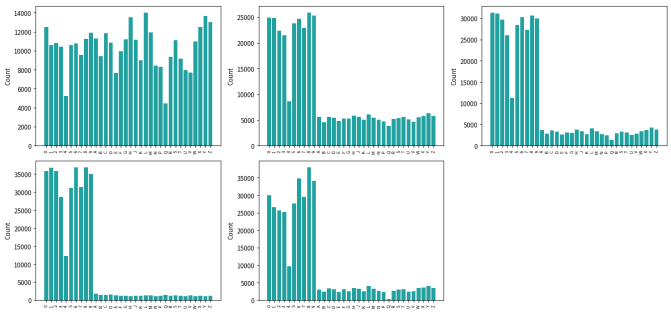

Fig. 4. Distribution of characters in positions 3 to 7

V. Experimental Setup

Following the KDD process, the experimental phase covers data preparation, model training under a stratified cross-validation protocol, and a final evaluation on data never used during model selection. All experiments use fixed random seeds for reproducibility, and the two pipeline stages are trained and validated on disjoint data preparations, described below.

A. Data Preparation and Sampling

Since the two stages consume different inputs (full scenes vs. plate crops), we derive two working sets from CCPD via the same stratified sampling procedure: images are drawn proportionally to subset size, with a minimum per-subset floor so that the rare adverse-condition subsets remain represented in every split.

1) Detection set: For the localization stage we sampled 6,800 full-resolution images (floor of 50 per subset). The corner annotations encoded in each CCPD filename were converted to the YOLO pose format: one object per image, with its axis-aligned bounding box and four keypoints reordered into a fixed top-left, top-right, bottom-right, bottom-left sequence, since a consistent corner ordering is required both by the keypoint loss and by the downstream homography.

2) Recognition set: For the recognition stage we sampled 68,000 images (floor of 100 per subset) and extracted one plate crop from each using the *ground-truth* corners. Each crop is stored at 168×48 pixels. To isolate the contribution of rectification, every image is cropped under the two paradigms introduced in Section III-D: the axis-aligned bounding box (*box*) and the homography-rectified quadrilateral (*warp*).

B. Validation Protocol

Both stages follow the same two-level protocol, designed to keep component-level assessment and system-level evaluation on strictly disjoint data.

First, the entire working set (the *pool*) is partitioned into four stratified folds by round-robin assignment within each subset, so that every fold reproduces the per-condition composition of the pool. Each model configuration is trained four times, holding out one fold as the *test* fold each time, and we report the mean and standard deviation of the metrics across the four held-out folds. No data is reserved as a separate frozen test set: every image belongs to exactly one held-out fold, so the cross-validation estimate is computed over the whole pool. For deployment, the selected configuration is then retrained on the full pool (with a 10% stratified slice retained for early stopping) to produce the weights used in the end-to-end pipeline.

Finally, to measure true end-to-end performance, we built a *fresh evaluation subset* of 5,000 images sampled from CCPD under the explicit constraint of excluding every image used in either stage's pool. On this subset the full pipeline is run exactly as deployed: YOLO11s-pose corners, homography rectification, and recognition, with no ground-truth information entering the chain. This protocol separates the questions "how well does each component perform in isolation?" (cross-validation) and "how well does the system perform as a whole, including error propagation between stages?" (fresh subset).

C. Training Configuration

1) Localization: All YOLO11 variants are trained at 640×640 input resolution with batch size 8 for 40 epochs per fold (50 for the final retrain), with early stopping (patience 15). Horizontal and vertical flip augmentations are disabled: mirroring a plate both inverts the corner ordering and produces unreadable mirrored text, making these standard augmentations harmful for this task. We compare COCO-pretrained initialization against training from scratch across the nano, small, and medium capacity.

2) LPRNet: LPRNet is trained with AdamW (learning rate 10^{-3}, weight decay 10^{-4}) under a cosine-annealing schedule, batch size 128, CTC loss with gradient-norm clipping at 5.0, for 30 epochs per fold and 40 in the final retrain; model selection within a run tracks the validation mean edit distance. Training crops are augmented with brightness jitter ($\pm 15\%$) and small random translations (± 2 px).

3) 7-head CNN: The multi-head classifier is trained with the sum of seven per-head cross-entropy losses, batch size 128, on the same crops with photometric-only augmentation (brightness and additive intensity jitter), since its fixed-slot design is, by construction, less tolerant of geometric jitter than the CTC decoder.

4) PaddleOCR: PP-OCRv5 is used strictly zero-shot, with no fine-tuning; its only adaptation is the output post-processing described in Section III-E.

D. Evaluation Metrics

For localization we report pose mAP@0.5 and mAP@0.5:0.95 on the held-out test folds, and, on the fresh subset, the *corner localisation error*: the mean Euclidean distance in pixels between predicted and ground-truth corners, which measures exactly the quantity that determines rectification quality, unlike box-level IoU. For recognition, the primary metric is *full-plate accuracy* (the fraction of plates whose entire 7-character string is correct), which we complement with per-position accuracy and the mean Levenshtein edit distance, to distinguish recognisers that fail catastrophically from those that miss single characters. End-to-end accuracy on the fresh subset is reported both overall and per CCPD subset. Computational cost is reported as inference throughput (FPS) measured on the development machine (NVIDIA GeForce RTX 2060, 6 GB VRAM).

VI. Results

The experimental results are presented in three parts. First, we evaluate localization performance across YOLOv11 variants and model sizes. Second, we analyze the impact of perspective rectification on the recognition stage. Third, we report the end-to-end performance of the complete pipeline on a fresh subset of CCPD images.

A. Localization: YOLOv11 Performance Analysis

We compared YOLOv11 *Segmentation* and *Pose Estimation* variants across three model scales: Nano (n), Small (s), and Medium (m).

TABLE II
Comparison of YOLOv11 architectures.

Model	mAP@50-95	Precision	Recall	MCE
YOLOv11n(Pose)	0.99351	0.99883	0.99736	4.22 px
YOLOv11s (Pose)	0.99426	0.99826	0.99876	3.79 px
YOLOv11m (Pose)	0.99366	0.99902	0.99850	4.19 px
YOLOv11n (Seg)	0.886855	0.99563	0.995528	7.44 px
YOLOv11s (Seg)	0.883010	0.99816	0.996322	7.93 px
YOLOv11m (Seg)	0.886855	0.99831	0.996188	8.06 px

B. Analysis of Perspective Correction

Before proceeding with a detailed performance analysis of the OCR systems, it is essential to evaluate the impact of perspective transformation (warping) on recognition quality. In real-world scenarios, license plates are captured by cameras at sub-optimal angles, resulting in distorted and perspectively altered images.

To demonstrate the necessity of this pre-processing step, we conducted a preliminary experiment comparing results on a subset of original (unwarped) images against the same images subjected to perspective correction, utilizing the vertex coordinates extracted by our architecture. This operation established an essential standard to ensure the maximum accuracy of the overall system.

The improvement in recognition quality comes at negligible runtime cost. Measured per plate on the same images, the perspective warp (`cv2.getPerspectiveTransform` followed by `cv2.warpPerspective`) required 0.090 ms on average, against 0.039 ms for a plain axis-aligned bounding-box crop — an additional overhead of approximately 0.05 ms per plate. This cost is several orders of magnitude smaller than the recognition step itself and well below the budget of any real-time deployment, making rectification effectively free for the substantial accuracy gain it provides, especially for our 7-head CNN.

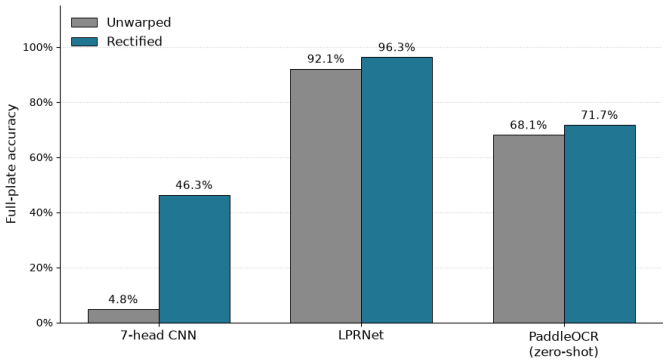

Fig. 5. Full-plate accuracy of each recogniser on unwarped vs rectified plate crops

C. Recognition on Rectified Crops

Following the rectification stage, we evaluated all three recognition models on the perspective-warped images. PaddleOCR serves as a zero-shot baseline drawn from a general-purpose Chinese OCR system, while LPRNet and the 7-head CNN are our custom, lightweight, domain-specific implementations. Table III summarizes the recognition accuracy, averaged across the held-out cross-validation folds.

The LPRNet architecture demonstrated the most significant resilience. While the 7-head CNN proved efficient, it was inherently tied to a rigid plate structure, whereas the LPRNet's CTC-based decoding proved more robust against minor character misalignments occurring after rectification.

TABLE III
OCR ENGINE PERFORMANCE (EVALUATED ON RECTIFIED CROPS).

Model	Full-Plate Acc.	Mean Edit Dist.
PaddleOCR (zero-shot)	0.717	0.569
7-head CNN	0.462	0.891
LPRNet	0.963	0.053

D. End-to-End Pipeline Evaluation

To measure the performance of the full pipeline under realistic conditions, we evaluated each recogniser on the fresh subset of 5,000 CCPD images, excluded from any prior training or validation. Plates were localised by the chosen YOLOv11s-Pose detector, rectified using the predicted corner coordinates, and then transcribed by each recogniser. Notably, YOLOv11s-Pose successfully detected the plate in every image of the fresh subset, with a median corner localization error of 3.5 px and a mean of 5.1 px.

Figure 6 reports the end-to-end full-plate accuracy of each recogniser, broken down by CCPD subset and ordered top-to-bottom from easiest (`Base`) to hardest (`Blur`). The breakdown reveals three distinct behavioural patterns. LPRNet remains above 84% on every subset and achieves perfect or near-perfect accuracy on `Base`, `Rotate`, and `Weather`, confirming the architecture's robustness across acquisition conditions. The 7-head CNN exhibits the sharpest degradation, dropping from 66% on `Base` to below 20% on all subsets other than `Weather`.

Averaged across the fresh subset, the three recognizers achieve full-plate accuracies of 95.8% (LPRNet), 71.6% (PaddleOCR), and 42.1% (7-head CNN). The gap between LPRNet's cross-validation result (96.3%) and its end-to-end result (95.8%) is small, about 0.5 pp, and quantifies the cost of imperfect upstream corner prediction. PaddleOCR is essentially unaffected by the transition to predicted corners, consistent with a general-purpose OCR engine that already tolerates mild geometric distortion, whereas the 7-head CNN shows the largest end-to-end gap (46.2% to 42.1%, -4.1 pp), reflecting its greater sensitivity to small misalignments in the rectified crops.

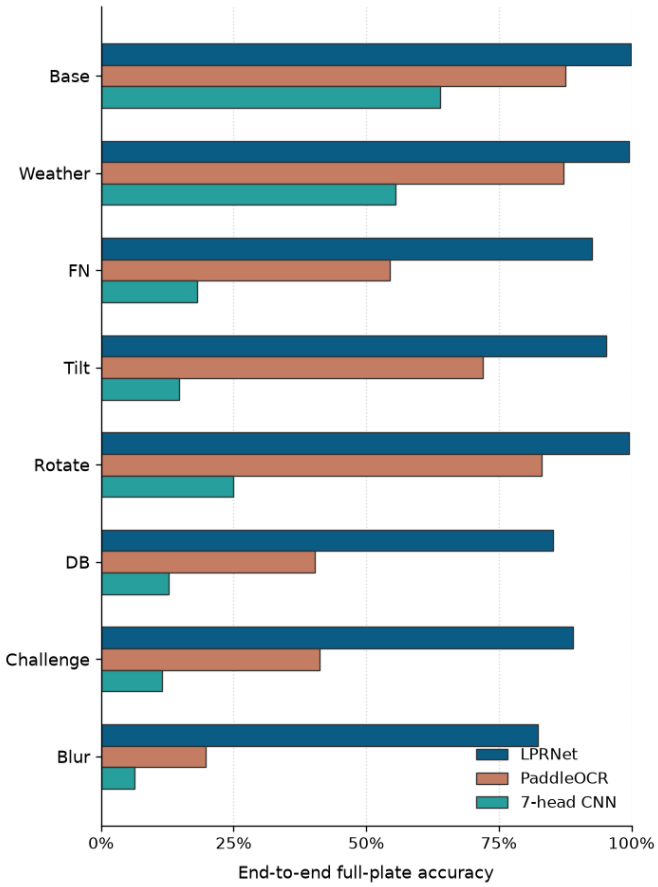

Fig. 6. End-to-end full-plate accuracy by CCPD subset

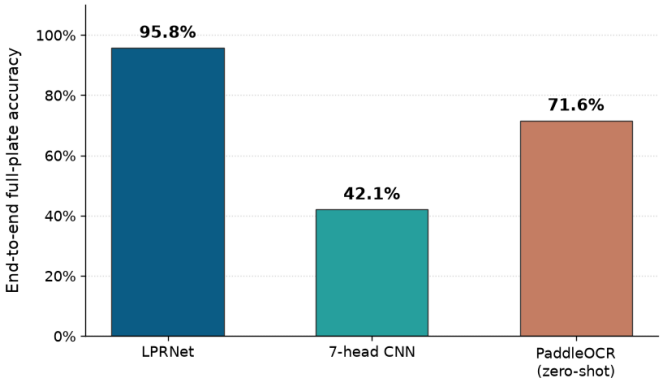

Fig. 7. End-to-end full-plate accuracy

VII. CONCLUSIONS

In this work we presented a complete two-stage ALPR pipeline for the CCPD benchmark, decoupling plate localization from character recognition so that the two central design axes could be evaluated independently under a stratified 4-fold cross-validation protocol and a final end-to-end test on a fresh, strictly disjoint subset.

For localization, the keypoint-regression (*pose*) formulation of YOLO11 clearly outperformed instance segmentation: all three pose scales exceeded 0.993 mAP@50–95 with a mean corner error below 4.3 px, whereas the segmentation variants plateaued around 0.88 mAP and roughly doubled the corner error (7.4–8.1 px). Since corner accuracy directly governs rectification quality, we selected YOLOv11s-Pose, which sports

the best accuracy (0.99426 mAP@50–95) and localization error (3.79 px) among the tested models, as the deployed detector. On the fresh subset it localised every plate, with a median corner error of just 3.5 px.

Our CTC-based LPRNet port achieved the highest full-plate accuracy on rectified crops (96.3%), well ahead of the PaddleOCR baseline (71.7%) and of the 7-head CNN (46.2%). The advantage carried over to the deployed pipeline: end-to-end, LPRNet reached 95.8% full-plate accuracy and remained above 84% on every CCPD subset.

Perspective rectification improved the models accuracy, whilst not causing a large overhead: at ~ 0.05 ms of added cost per plate it lifted the 7-head CNN by 41.5% whereas the other models report a much smaller accuracy gain.

Overall, the combination of YOLOv11s-Pose, plate rectification and LPRNet yields an ALPR system suitable for both the static and continuous-monitoring scenarios targeted in this work. Promising directions for future work include fine-tuning the recognizers on the harder `Blur` and `Challenge` conditions and validating real-time throughput under the video-monitoring deployment.

REFERENCES

[1] M. Veres and M. Moussa, "Deep learning for Intelligent Transportation Systems: A survey of emerging trends," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 8, pp. 3152–3168, 2020.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[4] Z. Xu, W. Yang, A. Meng, N. Lu, H. Huang, C. Ying, and L. Huang, "Towards end-to-end license plate detection and recognition: A large dataset and baseline," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 255–271.

[5] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth, "From data mining to knowledge discovery in databases," *AI Magazine*, vol. 17, no. 3, pp. 37–54, 1996.

[6] S. Zherzdev and A. Gruzdev, "LPRNet: License plate recognition via deep neural networks," *arXiv preprint arXiv:1806.10447*, 2018.

[7] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, and H. Wang, "PP-OCR: A practical ultra lightweight OCR system," *arXiv preprint arXiv:2009.09941*, 2020.

[8] S. Du, M. Ibrahim, M. Shehata, and W. Badawy, "Automatic license plate recognition (ALPR): A state-of-the-art review," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 23, no. 2, pp. 311–325, 2013.

[9] D. Lavezzini, "Sviluppo di un sistema ALPR in tempo reale per dispositivi edge: Confronto e ottimizzazione di YOLOX, RT-DETR v2 e CCT," Master's thesis, Politecnico di Torino, 2025.